Atomic Connectivity Module Specification

Periodic Atomic Graphs, Robust Connectivity Rules, and State Catalogs for mdstats

mdstats

2026-07-12

Contents

Purpose and status

This document specifies the implemented public atomic-connectivity module

`mdstats/analysis/atomic_connectivity.py`

introduced in mdstats 0.9.0. The API and invariants in this document are normative for the implemented module.

The module converts atomistic coordinates or explicit bond data into reproducible periodic atomic graphs. It also compresses repeated graphs across frames into connectivity states, trajectory segments, and edge transitions.

The central question is:

> Under one explicit scientific rule, which canonical atoms are connected in each frame?

Atomic connectivity is the explicit atom-level graph that later modules consume. It is not yet a framework abstraction. Framework roles, linker contraction, primitive rings, ring sites, cages, and dynamic spatial regions remain separate layers.

The intended architecture is

```
AtomisticFrameCollection
          |
          v
persistent ConnectivityScope
          |
          v
geometric pairs or explicit bonds
          |
          v
AtomicConnectivityDefinition
          |
          v
canonical AtomicConnectivityState objects
          |
          v
AtomicConnectivityResult
          |
          +--> framework_topology.py
          +--> topology_catalog.py
          +--> future connectivity observables
```

Design motives

A radial cutoff is often used informally as a bond definition. That is useful, but it becomes ambiguous when trajectories contain thermal noise, when ensembles are independently wrapped, or when a reference network is expected to survive moderate distortion.

The module makes the definition explicit and auditable. It supports:

- instantaneous distance connectivity;
- two-cutoff hysteresis for ordered trajectories;
- reference-based retention and formation rules for trajectories or ensembles;
- explicit externally supplied edge sets;
- fixed atom-identity scope;
- periodic edge image shifts;
- deterministic graph identity;
- repeated-state compression;
- exact atomic-edge additions and removals.

The design separates three scientific concepts:

geometric neighborhood discrete atomic connectivity projected framework topology .

RDF, radial coordination, and cutoff-defined neighbor angles remain separate observables. Atomic connectivity may later support graph-degree and connectivity-defined angle analyses, but it does not silently replace their existing mathematical definitions.

Responsibility boundaries

Owned by `atomic_connectivity.py`

The module owns:

- persistent connectivity scope;
- supported connectivity-rule objects;
- periodic atomic-edge meaning;
- atomic graph construction;
- trajectory hysteresis;
- reference-based connectivity classification;
- canonical periodic gauge normalization;
- graph-state serialization and digest generation;
- uniform, partitioned, and per-frame state organization;
- trajectory connectivity segments;
- atomic-edge transition records;
- connectivity provenance and diagnostics.

Owned by other modules

Module	Responsibility
`_neighbors.py`	Minimum-image geometry, distances, vectors, image shifts, blocking, and safe cutoff validation
`cutoffs.py`	Immutable species-pair cutoff values and cutoff provenance
`rdf.py`	Pair-density statistics and RDF feature detection
`coordination.py`	Instantaneous radial coordination distributions
`bond_angle.py`	Instantaneous cutoff-defined neighbor-angle distributions
`framework_topology.py`	Framework atom roles, linker paths, projected edges, and projected graph validation
`topology_catalog.py`	Reconciliation of projected framework topologies across frames
`region_membership.py`	Dynamic geometric region, phase, attachment, and detachment labels
Ring modules	Primitive rings, incidence, geometry, sites, and cages

The atomic and projected graph layers both require periodic canonicalization, but for different authoritative objects:

- `atomic_connectivity.py` normalizes the atomic graph;
- `framework_topology.py` later normalizes the contracted decorated framework graph.

Several different atomic states may project to the same framework topology. For example, changing Na-O contacts need not change a Si/Al-O framework when Na is a spectator.

Terminology and normative language

The words **must**, **must not**, **should**, and **may** are normative.

- **Active atom**: an atom retained by the resolved connectivity scope.
- **Atomic edge**: one undirected periodic connection between two active atoms.
- **Raw edge**: a frame-local edge before periodic gauge normalization.
- **Canonical state**: a gauge-normalized, sorted, immutable periodic graph.
- **Connectivity definition**: the scientific rule that decides edge presence.
- **Connectivity state**: one unique canonical atomic graph.
- **Catalog mode**: equal canonical states are deduplicated across frames.
- **Per-frame mode**: every frame retains an independent state record.
- **Frame index**: positional index in `AtomisticFrameCollection`.
- **Frame ID**: user- or source-provided label in `collection.frame_ids`.

Mathematical conventions

Cell and coordinate convention

For frame t, lattice vectors are rows of

$$H_t = \begin{pmatrix} \mathbf{a}_t^T \\ \mathbf{b}_t^T \\ \mathbf{c}_t^T \end{pmatrix}.$$

For wrapped fractional coordinates $\mathbf{s}_{i,t}$,

$$\mathbf{r}_{i,t} = \mathbf{s}_{i,t} H_t.$$

The collection has a fixed atom population and fixed canonical atom indices. Cells and coordinates may vary by frame.

Periodic edge convention

A directed representation of an undirected periodic edge from atom i to an image of atom j uses integer image shift $\mathbf{m}_{ij}$:

$$\mathbf{d}_{ij} = \left(\mathbf{s}_j - \mathbf{s}_i + \mathbf{m}_{ij}\right)H.$$

`_neighbors.py` must return $\mathbf{d}_{ij}$, its norm, and $\mathbf{m}_{ij}$ from the same minimum-image operation.

Reversing the edge gives

$$(i, j, \mathbf{m}_{ij}) \equiv (j, i, -\mathbf{m}_{ij}).$$

Strict cutoff relation

All distance rules use

$$r_{ij} < r_{\mathrm{cut}}.$$

An atom exactly on the cutoff is excluded. The strict inequality must agree with RDF-derived cutoff provenance and the existing neighbor kernel.

Exact labeled graph equality

Two connectivity states are equal when their canonical labeled graph records are exactly equal. General graph isomorphism is not used.

The first implementation assumes compatible frames preserve:

- atom count;
- atom ordering;
- atomic number at every canonical index;
- periodic-axis flags.

Structures with different atom orderings require an explicit atom-identity map in a future layer.

Required dependency updates

Neighbor image shifts

`NeighborListResult` in `_neighbors.py` must gain

```
image_shifts: NDArray[np.int64]  # shape (n_pairs, 3)
```

For every pair index `p`, the following arrays must describe the same periodic image:

```
result.neighbor_indices[p]
result.vectors[p]
result.distances[p]
result.image_shifts[p]
```

Existing RDF, coordination, and bond-angle outputs must remain numerically unchanged.

The existing public or internal two-output minimum-image helper may remain for current callers. A new private helper may return both vectors and shifts.

Deep cutoff immutability

`PairCutoff` and `PairCutoffRegistry` must be deeply immutable before they are stored inside connectivity definitions.

Requirements:

- registry mappings cannot be changed after construction;
- metadata is defensively copied into read-only structures;
- canonical pair ordering is fixed;
- connectivity definitions retain immutable canonical cutoff records;

- result provenance cannot change after calculation.

A frozen dataclass containing a mutable dictionary is not sufficient.

Public API overview

Root-level exports should include the common user-facing objects:

```
from mdstats import (
    ConnectivityScope,
    DistanceConnectivity,
    HystereticDistanceConnectivity,
    ReferenceDistanceConnectivity,
    ExplicitConnectivity,
    AtomicConnectivityResult,
    compute_atomic_connectivity,
)
```

Detailed public types should remain available from `mdstats.analysis.atomic_connectivity`:

```
from mdstats.analysis.atomic_connectivity import (
    AtomicEdgeKey,
    AtomicConnectivityState,
    ConnectivityConsistency,
    ConnectivitySegment,
    ConnectivityTransition,
    build_atomic_connectivity_state,
)
```

Low-level geometry, gauge normalization, hashing, state cataloging, and temporal reduction remain private.

Connectivity scope

Public data structure

```
@dataclass(frozen=True, slots=True)
class ConnectivityScope:
    included_species: tuple[int, ...] | None = None
    included_atom_indices: tuple[int, ...] | None = None
    excluded_species: tuple[int, ...] = ()
    excluded_atom_indices: tuple[int, ...] = ()
```

The direct constructor uses atomic numbers. Convenience constructors may accept symbols or atomic numbers:

```
ConnectivityScope.all()

ConnectivityScope.from_selection(
    included_species=("Si", "Al", "O"),
    excluded_species=("Na",),
    included_atom_indices=None,
    excluded_atom_indices=None,
)
```

Resolution rule

Let I_S be atoms selected by included species, I_A explicitly included atom indices, X_S atoms selected by excluded species, and X_A explicitly excluded indices.

When neither inclusion is supplied, the initial set is all atoms. Otherwise,

$$I = (I_S \cup I_A) \setminus (X_S \cup X_A).$$

Exclusions always win.

This union rule allows a broad species selection plus explicit inclusion of a special atom. The resolved selection is sorted and fixed across every analyzed frame.

Resolved scope

```
@dataclass(frozen=True, slots=True)
class ResolvedConnectivityScope:
    atom_indices: NDArray[np.int64]
    atomic_numbers: NDArray[np.int32]
    canonical_key: tuple[Any, ...]
```

`ResolvedConnectivityScope` is public at module level but need not be exported at the package root.

All arrays must be copied, C-contiguous, sorted, and read-only.

Scope meaning

A connectivity scope is identity-based, not spatially dynamic. It must not remove an atom merely because the atom:

- leaves a slab;
- enters a liquid;
- detaches from a framework;
- crosses an interface.

Such changes are recorded through lost edges and later through `region_membership.py`.

A breakaway framework oxygen must remain in scope so that its connectivity loss is visible.

Periodic atomic-edge key

```
@dataclass(frozen=True, order=True, slots=True)
class AtomicEdgeKey:
    atom_i: int
    atom_j: int
    image_shift: tuple[int, int, int]
```

Canonical orientation

For distinct atoms, canonical orientation requires

$$i < j.$$

A raw record with reversed endpoints is mapped as

$$(j, i, \mathbf{m}) \mapsto (i, j, -\mathbf{m}).$$

First-release limitations

The data model reserves periodic edge shifts, but the first distance-based implementation operates inside the unique-minimum-image radius. Therefore it supports at most one generated edge for one unordered pair of distinct atom indices.

The first release must reject:

- zero-shift self-edges;
- nonzero self-image edges;
- multiple parallel periodic edges between the same atom pair.

These cases may be supported later with a stronger gauge-invariant edge correspondence model.

Connectivity-definition types

The public type annotation is a closed union, not an open plugin protocol:

```
AtomicConnectivityDefinition = (
    DistanceConnectivity
    | HystereticDistanceConnectivity
    | ReferenceDistanceConnectivity
```

```
    | ExplicitConnectivity
)
```

Arbitrary third-party subclasses are not a stable extension point in the first release.

Instantaneous distance connectivity

```
@dataclass(frozen=True, slots=True)
class DistanceConnectivity:
    cutoffs: PairCutoffRegistry
    scope: ConnectivityScope = field(
        default_factory=ConnectivityScope.all
    )
```

For a registered species pair $A - B$,

$$(i, j) \in E_f \Leftrightarrow r_{ij}^{(f)} < r_{AB}^{\text{cut}}.$$

Properties:

- stateless;
- valid for trajectories and ensembles;
- independent of frame order;
- susceptible to threshold flickering.

Only registered cutoff pairs define eligible edge types. If the scope contains Si, Al, and O but the registry contains only Si-O and Al-O, then Si-Si, Al-Al, Si-Al, and O-O edges are undefined and cannot form.

An unregistered pair type is not an error.

Hysteretic distance connectivity

```
@dataclass(frozen=True, slots=True)
class HystereticDistanceConnectivity:
    formation_cutoffs: PairCutoffRegistry
    breaking_cutoffs: PairCutoffRegistry
    scope: ConnectivityScope = field(
        default_factory=ConnectivityScope.all
    )
    initial_state: Literal[
        "formation_cutoff",
        "explicit_edges",
    ] = "formation_cutoff"
    initial_edges: tuple[AtomicEdgeKey, ...] | None = None
```

For every registered pair,

$$r_{AB}^{\text{form}} < r_{AB}^{\text{break}}.$$

For an edge absent in the previous analyzed frame,

$$(i, j) \in E_f \Leftrightarrow r_{ij}^{(f)} < r_{AB}^{\text{form}}.$$

For an edge already present,

$$(i, j) \in E_f \Leftrightarrow r_{ij}^{(f)} < r_{AB}^{\text{break}}.$$

The interval

$$r_{\text{form}} \leq r < r_{\text{break}}$$

retains the previous edge state and suppresses thermal flickering.

Hysteresis is valid only when:

- `collection.semantics == FrameSemantics.TRAJECTORY`;
- selected collection positions are strictly increasing;
- selected positions are contiguous;
- frame stride is one.

It must reject ensembles, reordered frames, duplicate frames, and sparse frame selection.

When `initial_state="explicit_edges"`, `initial_edges` is required. Every initial edge must lie within scope and use a species pair registered by both cutoff registries.

Reference distance connectivity

```
@dataclass(frozen=True, slots=True)
class ReferenceDistanceConnectivity:
    discovery_cutoffs: PairCutoffRegistry
    formation_cutoffs: PairCutoffRegistry
    retention_cutoffs: PairCutoffRegistry
    reference_frame: int = 0
    scope: ConnectivityScope = field(
        default_factory=ConnectivityScope.all
    )
```

Reference edges are discovered from one frame using

$$r_{ij}^{(\mathrm{ref})} < r_{AB}^{\mathrm{discover}}.$$

For every target frame:

- a reference pair is retained when

$$r_{ij} < r_{AB}^{\mathrm{retain}};$$

- a nonreference pair forms when

$$r_{ij} < r_{AB}^{\mathrm{form}}.$$

Recommended and required first-release ordering is

$$r_{AB}^{\mathrm{form}} \leq r_{AB}^{\mathrm{discover}} < r_{AB}^{\mathrm{retain}}.$$

Reference classification is independent of target-frame order. It is valid for trajectories and ensembles.

The reference frame is a collection positional index. It may lie outside the analyzed target-frame subset, but it must belong to the same collection.

Explicit connectivity

```
@dataclass(frozen=True, slots=True)
class ExplicitConnectivity:
    scope: ConnectivityScope = field(
        default_factory=ConnectivityScope.all
    )
    uniform_edges: tuple[AtomicEdgeKey, ...] | None = None
    frame_edges: Mapping[
        int,
        tuple[AtomicEdgeKey, ...],
    ] | None = None
```

Exactly one of `uniform_edges` and `frame_edges` must be supplied.

Keys in `frame_edges` are collection positional frame indices, not frame IDs.

Explicit connectivity supports:

- imported bond lists;

- known reference graphs;
- external bond-order classifiers;
- unit tests;
- future file-format topology.

All explicit edges are scope checked, species checked where a definition requires it, endpoint canonicalized, deduplicated, gauge normalized, and sorted.

Connectivity consistency

```python
class ConnectivityConsistency(Enum):
    UNIFORM = "uniform"
    PARTITIONED = "partitioned"
    PER_FRAME = "per_frame"
```

- `UNIFORM`: exactly one canonical state occurs in catalog mode.
- `PARTITIONED`: multiple canonical states occur and are reused in catalog mode.
- `PER_FRAME`: each analyzed frame owns an independent state record.

`PARTITIONED` does not depend on an arbitrary threshold for the number of states.

For trajectories, catalog mode also constructs contiguous state segments. For ensembles, frames are grouped by state without assigning temporal meaning to their stored order.

Atomic connectivity state

```python
@dataclass(frozen=True, slots=True)
class AtomicConnectivityState:
    active_atom_indices: NDArray[np.int64]
    active_atomic_numbers: NDArray[np.int32]
    pbc: NDArray[np.bool_]

    edge_atom_indices: NDArray[np.int64]
    edge_image_shifts: NDArray[np.int64]

    degree: NDArray[np.int32]
    component_labels: NDArray[np.int32]
    n_components: int

    canonical_schema_version: str
    digest_algorithm: str
    digest: str
```

Array shapes

Field	Shape
`active_atom_indices`	`(n_active,)`
`active_atomic_numbers`	`(n_active,)`
`pbc`	`(3,)`
`edge_atom_indices`	`(n_edges, 2)`
`edge_image_shifts`	`(n_edges, 3)`
`degree`	`(n_active,)`
`component_labels`	`(n_active,)`

All arrays must be copied, C-contiguous, and read-only.

`edge_atom_indices` uses global canonical atom indices, not positions within the active-atom array.

A connectivity state does not contain its local catalog ID. The containing result assigns dense state IDs through tuple position:

```
state = result.states[state_id]
```

Degree and components

For every supported edge between distinct atoms, both endpoint degrees increase by one.

Disconnected states are valid:

$$N_{\text{components}} \geq 1.$$

Connectedness is a later material-specific framework validation property, not an atomic-connectivity invariant.

Convenience methods

Recommended module-level public methods or properties are:

```
state.n_active_atoms
state.n_edges
state.edge_keys
state.degree_for_atom(atom_index)
state.to_networkx()
state.to_dict()
AtomicConnectivityState.from_dict(payload)
```

NetworkX is a derived view. It is not canonical storage and must not influence state identity. State equality compares the complete canonical arrays rather than NumPy object identity. Runtime hashing may be derived from the stable structural digest, but the digest remains the persistent identifier.

Periodic gauge normalization

Gauge freedom

Independent wrapping changes fractional representatives:

$$\mathbf{s}_i' = \mathbf{s}_i + \mathbf{g}_i, \qquad \mathbf{g}_i \in \mathbb{Z}^3.$$

To preserve the same physical edge displacement,

$$\boxed{\mathbf{m}_{ij}' = \mathbf{m}_{ij} + \mathbf{g}_i - \mathbf{g}_j}\,.$$

The sign convention must be tested through

$$\mathbf{s}_j' - \mathbf{s}_i' + \mathbf{m}_{ij}' = \mathbf{s}_j - \mathbf{s}_i + \mathbf{m}_{ij}.$$

Canonical gauge algorithm

For each connected component:

1. select the smallest active atom index as root;
2. assign the root gauge offset $\mathbf{g} = \mathbf{0}$;
3. construct a deterministic spanning tree from sorted unordered atom pairs;
4. for tree edge $(i, j, \mathbf{m}_{ij})$, propagate

$$\mathbf{g}_j = \mathbf{g}_i + \mathbf{m}_{ij};$$

5. transform every edge shift by

$$\mathbf{m}_{ij}^{\text{canon}} = \mathbf{m}_{ij} + \mathbf{g}_i - \mathbf{g}_j;$$

6. orient every edge with the smaller atom index first;
7. sort edge records lexicographically.

Every tree edge becomes zero-shift. Non-tree edge shifts retain the periodic cycle information.

Isolated active atoms receive zero gauge offsets.

The first implementation permits only one edge per unordered atom pair, making the deterministic spanning tree unambiguous after endpoint sorting.

Stable structural identity

Define

```
CANONICAL_CONNECTIVITY_SCHEMA = (
    "mdstats.atomic-connectivity.v1"
)
```

A structural state digest includes:

- canonical schema identifier;
- active atom indices;
- active atomic numbers;
- periodic-axis flags;
- canonical edge endpoint array;
- canonical edge image-shift array.

It must not include:

- instantaneous cell lengths or angles;
- Cartesian coordinates;
- cutoff provenance;
- frame order;
- state ID;
- package object identity.

A cryptographic stable digest such as SHA-256 or BLAKE2b must be used. Python's built-in `hash()` must not be used.

Digest equality is only a lookup optimization. Exact array equality must be verified before two states are declared equal.

Structural identity versus provenance

Two different connectivity definitions that produce the same canonical graph must produce the same structural state digest.

Analysis provenance is recorded separately and may have its own digest containing:

- connectivity-definition kind;
- scope declaration;
- cutoff values and provenance;
- reference frame;
- hysteresis initialization;
- package version and options.

This separation allows ring identity to depend on the actual graph rather than on how an equivalent graph was discovered.

Segments and transitions

Connectivity segment

```
@dataclass(frozen=True, slots=True)
class ConnectivitySegment:
    segment_id: int
    state_id: int
    result_position_start: int
    result_position_stop: int
```

The result-position interval is half-open:

$$[s, e).$$

For state sequence

```
A A A B B A A
```

there are two unique states and three segments.

Segments exist only for trajectories in catalog mode.

Connectivity transition

```python
@dataclass(frozen=True, slots=True)
class ConnectivityTransition:
    transition_id: int
    source_state_id: int
    target_state_id: int

    result_position_before: int
    result_position_after: int
    collection_frame_index_before: int
    collection_frame_index_after: int
    frame_id_before: int
    frame_id_after: int

    added_edges: tuple[AtomicEdgeKey, ...]
    removed_edges: tuple[AtomicEdgeKey, ...]
    affected_atom_indices: tuple[int, ...]
```

Transition matching in the first release uses unordered atom-pair identity:

$$P(E) = \{(\min(i, j), \max(i, j))\}.$$

Then

$$P_{\text{added}} = P(E_B) \setminus P(E_A),$$

$$P_{\text{removed}} = P(E_A) \setminus P(E_B).$$

This is necessary because independent gauge normalization may change the image shift of unchanged edges when the deterministic spanning tree changes.

Each added edge stores its target-state image shift. Each removed edge stores its source-state image shift.

Transitions are temporal and are returned only for trajectories in catalog mode. Ensembles use state grouping and explicit state comparison, not temporal transitions.

Collection-level result

```python
@dataclass(frozen=True, slots=True)
class AtomicConnectivityResult:
    definition: AtomicConnectivityDefinition
    resolved_scope: ResolvedConnectivityScope
    consistency: ConnectivityConsistency

    frame_indices: NDArray[np.int64]
    frame_ids: NDArray[np.int64]
    frame_state_ids: NDArray[np.int32]
    states: tuple[AtomicConnectivityState, ...]

    segments: tuple[ConnectivitySegment, ...] | None
    transitions: tuple[ConnectivityTransition, ...]

    metadata: Mapping[str, Any]
```

Result constraints

- `frame_indices`, `frame_ids`, and `frame_state_ids` have equal length;
- frame indices are valid collection positions;
- arrays are read-only;
- every frame-state ID indexes `states`;
- state IDs are deterministic after canonical state sorting;
- `segments is None` for ensembles and per-frame mode;
- `transitions == ()` for ensembles and per-frame mode;
- `UNIFORM` has exactly one state;
- `PARTITIONED` has at least two states;
- `PER_FRAME` has one state object per analyzed frame.

Recommended methods

```
result.n_states
result.state_counts
result.state_probabilities
result.state_for_frame(frame_index)
result.frames_for_state(state_id)
result.edge_presence(edge)
result.to_networkx(state_id)
result.to_dict()
AtomicConnectivityResult.from_dict(payload)
```

`state_for_frame()` accepts a collection positional frame index, not a frame ID.

Public functions

Collection-level calculation

```
def compute_atomic_connectivity(
    collection: AtomisticFrameCollection,
    definition: AtomicConnectivityDefinition,
    *,
    frame_indices: ArrayLike | None = None,
    state_mode: Literal["catalog", "per_frame"] = "catalog",
    atom_block_size: int = 256,
    neighbor_search_options: NeighborSearchOptions | None = None,
    verlet_cache_options: VerletCacheOptions | None = None,
) -> AtomicConnectivityResult:
    ...
```

Inputs

`collection` Fixed-population atomistic frame collection.

`definition` One supported immutable connectivity definition.

`frame_indices` Optional positional frame indices. `None` selects every frame. Duplicates are rejected. Order is retained for stateless definitions. Hysteresis imposes stricter ordering and contiguity requirements.

`state_mode` `"catalog"` deduplicates equal canonical states. `"per_frame"` retains one independent state per analyzed frame.

`atom_block_size` Positive integer controlling blocked dense pair geometry when the dense backend is used.

`neighbor_search_options` Optional production `NeighborSearchOptions`. One analysis-local executor serves distance, hysteretic, and reference candidate requests. Automatic Verlet reuse is limited to eligible time-ordered trajectories; single-frame selections and independent ensembles are stateless unless an expert explicitly requests caching. Geometric execution is backend-neutral; connectivity owns persistent bond state.

`verlet_cache_options` Compatibility alias for forcing the cell-list/Verlet path with S2/S3 options. New code should use `neighbor_search_options`. Passing both option objects is an error.

Output behavior

In catalog mode:

- one unique state gives `UNIFORM`;
- multiple unique states give `PARTITIONED`;
- equal states are stored once;
- trajectory segments and transitions are created.

In per-frame mode:

- `states` has one entry per analyzed frame;
- `frame_state_ids == arange(n_frames)`;
- `consistency == PER_FRAME`;
- no cross-frame segment or transition promise is made.

Single-frame construction

```
def build_atomic_connectivity_state(
    collection: AtomisticFrameCollection,
    definition: DistanceConnectivity | ExplicitConnectivity,
    *,
    frame_index: int,
    atom_block_size: int = 256,
) -> AtomicConnectivityState:
    ...
```

The helper supports only definitions that are meaningful without cross-frame context.

Hysteretic connectivity must use `compute_atomic_connectivity()`.

Reference connectivity initially remains collection-level because reference preparation is shared across target frames.

Core algorithms

Scope resolution

1. validate species and indices;
2. resolve inclusion union;
3. apply exclusions;
4. sort global canonical indices;
5. copy corresponding atomic numbers;
6. freeze arrays;
7. construct a canonical scope key.

Scope resolution is performed once per call, not once per frame.

Frame-local distance edge generation

For every canonical pair in the cutoff registry:

1. intersect each species group with the resolved active atoms;
2. skip pair types with an empty endpoint group and record a diagnostic;
3. call the shared neighbor path at the pair cutoff, using the optional request-keyed session when configured;
4. use unordered-identical counting when both species are the same;
5. otherwise evaluate one directed $A \to B$ pair search;
6. convert accepted pairs to global atom indices and integer image shifts;
7. canonicalize endpoint orientation;
8. deduplicate exact raw records;
9. reject unsupported self-image or parallel edges;
10. normalize periodic gauge;
11. construct the immutable state.

If no registered cutoff pair is eligible in the resolved scope, the call must fail clearly because the distance definition cannot evaluate any edge type.

An eligible definition may still produce an empty edge set in a particular frame. The resulting state is valid and contains isolated active atoms.

High-temperature topology classification

For thermally fluctuating trajectories, topology classification should use `HystereticDistanceConnectivity` rather than one threshold applied independently to every frame. Formation and breaking cutoffs must be reported in provenance, with $r_{\mathrm{form}} < r_{\mathrm{break}}$. Data-driven cutoff estimation is an adapter or example responsibility; the connectivity module owns only validation and stateful bond updates. Persistent topology classes are then constructed by `topology_catalog.py`.

Hysteresis update

Initialization with `formation_cutoff` evaluates the first selected frame using formation cutoffs.

For every later selected frame:

1. identify currently present unordered atom pairs;
2. evaluate each present pair using its breaking cutoff;
3. retain present pairs satisfying $r < r_{\mathrm{break}}$;
4. search eligible absent pairs using formation cutoffs;
5. add absent pairs satisfying $r < r_{\mathrm{form}}$;
6. recalculate each accepted pair's current minimum-image shift;
7. canonicalize the complete graph;
8. catalog or store the state.

Present pairs must not be reconsidered as formation candidates in the same frame.

Trajectory fractional coordinates may be unwrapped, but graph identity must still be canonicalized because input reconstruction and cell changes can alter raw image labels.

Reference classification

Reference preparation:

1. resolve scope;
2. evaluate the reference frame at discovery cutoffs;
3. store the reference unordered atom-pair set;
4. retain the canonical reference state.

For each target frame:

1. evaluate every reference pair directly;
2. retain it when $r < r_{\mathrm{retain}}$;
3. search nonreference candidates at formation cutoffs;
4. exclude pairs belonging to the reference-pair set;
5. combine retained and newly formed pairs;
6. calculate current image shifts;
7. canonicalize and catalog the graph.

Target-frame order must not affect results.

State catalog construction

In catalog mode:

1. calculate the canonical state digest;
2. find existing states with the same digest;
3. verify exact array equality;
4. reuse the state on an exact match;
5. otherwise register a new unique state;
6. after all frames, sort unique states by their full canonical structural key;

7. remap frame-state IDs to the deterministic sorted order;
8. build trajectory segments and transitions.

State IDs must not depend on frame encounter order.

Input constraints

Collection constraints

The collection must have:

- at least one frame;
- fixed atom count;
- fixed atomic numbers and atom ordering;
- finite cells and coordinates;
- valid three-component PBC mask;
- frame semantics declared as trajectory or ensemble.

Scope constraints

The resolved scope must:

- be nonempty;
- contain only valid canonical atom indices;
- remain fixed across selected frames;
- have deterministic sorted order.

Cutoff constraints

Every cutoff must be:

- finite;
- positive;
- inside the neighbor kernel's safe periodic range for every selected frame where it is used.

For multi-registry definitions, canonical pair-key sets must agree exactly.

Hysteresis requires

$$r_{\mathrm{form}} < r_{\mathrm{break}}.$$

Reference connectivity requires

$$r_{\mathrm{form}} \leq r_{\mathrm{discover}} < r_{\mathrm{retain}}.$$

Frame-selection constraints

For stateless and reference definitions:

- valid, unique frame positions are required;
- user order may be retained;
- ensemble order has no temporal interpretation.

For hysteresis:

- trajectory semantics are required;
- selected positions must be consecutive and strictly increasing;
- unit stride is required.

Errors and diagnostics

Public exceptions

The module should define public module-level exceptions:

```
class AtomicConnectivityError(ValueError): ...
class ConnectivityScopeError(AtomicConnectivityError): ...
class ConnectivityDefinitionError(AtomicConnectivityError): ...
class ConnectivityFrameSelectionError(AtomicConnectivityError): ...
class ConnectivityGeometryError(AtomicConnectivityError): ...
```

They need not be exported at the package root.

Required errors

The implementation must reject:

- empty or malformed scope;
- invalid or duplicate explicit atom indices;
- nonfinite geometry;
- malformed explicit edges;
- explicit edges outside scope;
- zero-shift self-edges;
- unsupported self-image or parallel edges;
- unsafe periodic cutoffs;
- incompatible cutoff registries;
- no eligible registered pair type in a distance definition;
- invalid cutoff ordering;
- hysteresis on an ensemble;
- hysteresis on reordered, duplicate, or sparse frames;
- inconsistent atom population or atom identity;
- unsupported connectivity-definition objects.

Diagnostics and warnings

The result metadata should record nonfatal diagnostics such as:

- registered pair types with empty endpoint groups;
- ambiguous equal-distance periodic-image ties;
- threshold-marginal edges when requested;
- rapid recurrence of graph states;
- unusually many unique states relative to frame count;
- one-edge changes;
- empty edge states.

Disconnected states are not automatically warnings. Multiple components are valid for molecular, heterogeneous, fractured, or multi-slab systems.

Spectator-driven changes are not diagnosed here because spectator roles do not exist until framework projection.

Provenance and serialization

`AtomicConnectivityResult.metadata` must include enough information to reproduce the calculation:

- source format and source files when available;
- package version;
- selected collection frame indices and frame IDs;
- frame semantics;
- resolved connectivity scope;
- definition kind and canonical definition record;
- all cutoff values and provenance;
- strict cutoff inequality;
- reference frame where applicable;
- hysteresis initialization policy;
- canonical schema version;

- digest algorithm;
- periodic gauge convention;
- unique-image limitation;
- neighbor atom block size;
- optional neighbor-cache statistics;
- diagnostics.

Serialization must not rely on Python object addresses or unordered dictionary iteration.

Round-trip reconstruction must reproduce exact structural keys and digests.

Memory and performance model

The module should store unique states rather than one complete edge set per frame in catalog mode.

For F analyzed frames and K unique states,

$$\text{storage} \sim O(F) + O\left(\sum_{k=1}^{K} \mid E_k \mid\right).$$

For a uniform zeolite trajectory, $K = 1$. For one irreversible bond-breaking event, K is commonly two.

Without cache options, connectivity retains the blocked dense path. With cache options, each exact species-pair request is rebuilt through the S1 cell list at cutoff plus skin and then reused under either the default S2 fixed-cell bound or the explicit S3 deformation-aware bound. Neither path changes the connectivity-state contract.

Distances and displacement vectors are not retained in connectivity states. They are recomputed when later geometry analyses require them.

Interactions with later modules

Framework topology

`framework_topology.py` consumes one `AtomicConnectivityState` and one explicit `FrameworkMapping`.

The mapping later assigns:

```
VERTEX
LINKER
SPECTATOR
EXCLUDED
```

and contracts accepted atomic paths into decorated framework edges.

Atomic connectivity must not assign these roles.

Topology catalog

`topology_catalog.py` projects each referenced atomic state once in catalog mode, then deduplicates projected framework topologies by the exact Stage 2 structural key. Multiple atomic states may produce one framework topology; the catalog retains state-level provenance and frame assignments.

Region membership

`region_membership.py` may consume connectivity components to identify attached solid backbones and detached fragments. It must not alter the persistent connectivity scope or rewrite graph-state identity.

Existing structural observables

Stage 1 must not change the scientific definitions of:

```
compute_pair_rdf
compute_coordination_distribution
compute_bond_angle_distribution
```

Future connectivity-specific observables should be separately named, for example:

```
compute_connectivity_degree_distribution(...)
compute_connectivity_angle_distribution(...)
```

Usage examples

Narrow zeolite framework scope

```
scope = ConnectivityScope.from_selection(
    included_atom_indices=initial_framework_atom_indices,
)

definition = DistanceConnectivity(
    cutoffs=PairCutoffRegistry.from_cutoffs(
        [si_o_cutoff, al_o_cutoff]
    ),
    scope=scope,
)

result = compute_atomic_connectivity(
    collection,
    definition,
)
```

This preserves the identity of the initial framework atoms even if one later moves into a liquid region.

Hysteretic trajectory connectivity

```
definition = HystereticDistanceConnectivity(
    formation_cutoffs=form_cutoffs,
    breaking_cutoffs=break_cutoffs,
    scope=scope,
)

result = compute_atomic_connectivity(
    trajectory,
    definition,
    state_mode="catalog",
    verlet_cache_options=VerletCacheOptions(skin=0.6),
)
```

The hysteretic implementation evaluates the breaking cutoff once per pair type and frame, then derives the formation subset from the same current distances. Bond-history state remains outside the geometric cache.

Reference-based ensemble connectivity

```
definition = ReferenceDistanceConnectivity(
    discovery_cutoffs=discovery_cutoffs,
    formation_cutoffs=formation_cutoffs,
    retention_cutoffs=retention_cutoffs,
    reference_frame=0,
    scope=scope,
)

result = compute_atomic_connectivity(
    ensemble,
    definition,
)
```

The ensemble frame order does not influence classification.

Broad reusable connectivity

```
scope = ConnectivityScope.from_selection(
    included_species=("Si", "Al", "O", "Na", "Li", "K"),
)
```

This may retain guest-framework contacts. A later framework mapping must still mark Na, Li, and K as spectators so they cannot become projected framework vertices or linkers.

Edge cases and limitations

Thermal cutoff flicker

Instantaneous distance connectivity may alternate rapidly near one cutoff. Use hysteresis for a trajectory or reference-based connectivity for independently sampled structures.

Sparse trajectory sampling

Hysteresis on sparse frames can miss intermediate formation or breaking events. The first release rejects sparse selection rather than silently applying an ambiguous state history.

Different atom ordering

Equal composition does not imply compatible atom identity. This module does not solve graph isomorphism or atom remapping across independently reordered files.

Dynamic spatial boundaries

A fixed connectivity scope is not a slab or phase classifier. Atoms that leave a solid remain in scope. Dynamic spatial classification is deferred to `region_membership.py`.

Large cutoffs and multiple images

The first release requires unique minimum images and rejects unsupported parallel or self-image edges. Very small cells or large cutoffs may violate this condition.

Broad scopes

A broad scope may produce many atomic states from mobile-species contact changes even while the structural framework remains unchanged. This is expected. The framework-topology layer later removes spectators and may deduplicate those states into one projected topology.

Reference bias

Reference connectivity is robust but reference dependent. A poor reference frame or unjustified retention radius may hide meaningful structural change or label a distorted bond as broken. The reference and all thresholds must be reported.

Test specification

Neighbor-kernel integration

Tests must verify:

- orthogonal image shifts;
- triclinic image shifts;
- mixed periodic axes;
- boundary-crossing pairs;
- strict cutoff exclusion at equality;
- unchanged RDF, coordination, and bond-angle regression results.

Scope

Tests must verify:

- all-atom scope;
- species inclusion;

- atom-index inclusion;
- union of inclusion sources;
- exclusion precedence;
- exact initial framework index scope;
- broad scope containing mobile ions;
- explicit edge rejection outside scope.

Edge identity and canonicalization

Tests must verify:

- endpoint reversal maps $\mathbf{m} \to -\mathbf{m}$;
- raw duplicates collapse;
- deterministic edge sorting;
- invalid self-edges fail;
- unsupported parallel edges fail.

Periodic gauge invariance

Tests must verify:

- independent atom wrapping preserves state identity;
- lattice translation of one connected component preserves identity;
- candidate-pair order does not affect the state;
- atom block size does not affect the state;
- the displacement reconstruction identity fixes the gauge sign convention.

Distance connectivity

Tests must verify:

- uniform graph across frames;
- edge appearance and disappearance;
- identical-species pairs counted once;
- disconnected states;
- isolated active atoms;
- empty edge state with eligible pair definitions.

Hysteresis

Tests must verify:

- absent edge does not form inside the hysteresis interval;
- present edge does not break inside the interval;
- formation occurs below the formation cutoff;
- breaking occurs at or above the breaking cutoff;
- recurring A-B-A behavior yields two states and three segments;
- ensemble input fails;
- reordered, duplicate, and sparse frame selection fail.

Reference connectivity

Tests must verify:

- distorted but uniform ensemble;
- reference edge retained beyond discovery cutoff;
- reference edge removed beyond retention cutoff;
- new edge formed below formation cutoff;
- target-frame permutation does not change state identities;
- reference frame may lie outside the target subset.

Cataloging and transitions

Tests must verify:

- one unique state gives `UNIFORM`;
- several reused states give `PARTITIONED`;
- per-frame mode gives `PER_FRAME`;
- state IDs are deterministic after canonical sorting;
- transition pair matching is gauge robust;
- added and removed edge records use target and source shifts respectively;
- ensembles return no temporal segments or transitions.

Persistence

Tests must verify:

- stable digest across processes;
- exact dictionary round trip;
- digest equality followed by exact structural comparison;
- installed-wheel smoke behavior;
- specification signatures match implementation signatures.

Deferred functionality

The first release deliberately defers:

- ensemble-consensus connectivity;
- bond-order connectivity;
- callback/plugin connectivity rules;
- transition persistence or debouncing beyond hysteresis;
- parallel periodic edges;
- nonzero self-image edges;
- atom-identity remapping;
- dynamic region membership;
- framework role assignment and linker projection;
- connectivity-defined coordination and angle public APIs;
- ring analysis.

These features must be added only with explicit scientific definitions and focused tests.

Implementation organization

Layer 1: geometry and immutable dependencies

- add neighbor image shifts;
- harden `PairCutoff` and `PairCutoffRegistry` immutability;
- preserve existing structural-module behavior.

Layer 2: scope and edge foundation

- implement scope normalization;
- implement `AtomicEdgeKey`;
- validate and canonicalize raw edges.

Phase 1C: canonical state foundation

- implement connected components and degree;
- implement periodic gauge normalization;
- implement immutable state arrays;
- implement canonical serialization and stable digest.

Phase 1D: stateless definitions
- implement distance connectivity;
- implement explicit connectivity;
- implement single-frame state construction.

Phase 1E: state result and catalog
- implement state deduplication;
- assign deterministic state IDs;
- build frame-state mapping;
- build trajectory segments and transitions.

Phase 1F: robust multi-frame rules
- implement hysteretic trajectory connectivity;
- implement reference-based trajectory and ensemble connectivity.

Phase 1G: integration and release
- add public exports;
- add this specification to package distributions;
- update architecture-manual references and changelog;
- run focused and full regression tests;
- build wheel and source distributions;
- test the installed wheel.

Final invariants
The implementation must preserve the following invariants:

1. `AtomisticFrameCollection` remains connectivity agnostic.
2. Connectivity scope is fixed and identity based.
3. `_neighbors.py` owns geometry but does not define bonds.
4. Registered pair cutoffs define allowed distance-edge types.
5. Hysteresis depends only on ordered trajectory history.
6. Ensemble classification never depends on arbitrary frame order.
7. State identity is exact labeled periodic graph equality.
8. Structural identity is separate from analysis provenance.
9. Atomic and projected framework graphs own separate canonicalization layers.
10. State IDs belong to the containing result, not to immutable state objects.
11. Transition matching is robust to independent gauge normalization.
12. Existing RDF, radial coordination, and neighbor-angle definitions remain unchanged.
13. Dynamic region membership cannot alter connectivity scope.
14. Same active atom identities, PBC, and canonical periodic graph imply the same structural connectivity state.

The Stage 1 implementation is complete only when these invariants are enforced by both code and tests.

S4 periodic-neighbor integration
Distance, hysteretic, and reference connectivity obtain geometric candidates from the shared exact S4 subsystem. Formation and breaking classification use one candidate-distance pass per frame. Hysteretic state, reference eligibility, gauge normalization, cataloging, and transitions remain atomic-connectivity responsibilities. Cache rebuild logic is not part of connectivity identity.

Execution diagnostics are stored at `metadata["neighbor_search"]`. A `neighbor_cache` metadata alias is retained when cache statistics exist. The normative backend and fallback contract is `docs/specs/analysis/neighbor_search_spec.md`.

LTA framework-topology hysteresis example

The packaged LTA framework/density example uses `HystereticDistanceConnectivity` for Si–O and Al–O topology classification. The topology catalog is built from a **framework-only** connectivity result; mobile Li/Na/K–O contacts are excluded from the source state identity and are added only to the separate atomic mean-connectivity calculation.

For each present tetrahedral species, the example samples the four nearest oxygen distances over at most 96 evenly spaced frames. Formation and breaking cutoffs are calibrated separately, with the invariant

$$r_{\mathrm{form}} < r_{\mathrm{break}}.$$

An existing bond remains active while $r < r_{\mathrm{break}}$, even when a thermal excursion places it above r_{form}. This suppresses one-frame topology fragmentation caused by ordinary bond stretching. The resolved pair cutoffs and sampled quantiles are emitted as audit metadata. Explicit command-line overrides must still satisfy the strict formation-before-breaking inequality.